QALight

Frontend-розробка

Заняття 27. Інтерфейси

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Створення та використання інтерфейсу. Readonly властивості. Інтерфейси та псевдоніми типів. Розширення та реалізації інтерфейсів. Створення інтерфейсу.

Інтерфейси

Створення та використання інтерфейсу. Readonly властивості. Інтерфейси та псевдоніми типів. Інтерфейси класів. Розширення та реалізації інтерфейсів.

Створення типу об'єкта з використанням type

Теорія: TypeScript

У TypeScript створити тип об'єкта User та використати його як аргумент функції можна за допомогою ключового слова **type** або **interface**.

```
// Створення типу об'єкта User
type User = {
 id: number;
 name: string;
 email: string;
 age?: number; // Знак '?' означає необов'язкове поле
};
```

Використання типу User як аргументу функції

Теорія: TypeScript

```
// Використання типу User як аргументу функції
function printUserInfo(user: User): void {
 console.log(`Користувач: ${user.name}, Email: ${user.email}`);
}
// Приклад виклику функції
const currentUser: User = {
 id: 1,
 name: 'Олена',
 email: 'olena@example.com',
 age: 28,
};
printUserInfo(currentUser);
```

Інтерфейси є основною концепцією TypeScript

Теорія: TypeScript

Інтерфейси є основною концепцією TypeScript і широко використовуються для визначення форм об’єктів.
Вони в основному використовуються для оголошення контрактів, надаючи спосіб визначення форми об’єкта, якого має дотримуватися клас або інші об’єкти.

```
// Interface usage
interface Person {
 name: string;
 age: number;
}
const person: Person = {
 name: "John",
 age: 30,
};
```

Інтерфейси можна розширювати один від одного (extends), дозволяючи створювати більш спеціалізовані інтерфейси на основі існуючих.

Вони підтримують об’єднання декларацій, тобто ви можете оголосити інтерфейс у кількох місцях, а TypeScript об’єднає їх у єдине визначення.

Створення типу об'єкта з використанням interface

Теорія: TypeScript

Для об'єктів у TypeScript також часто використовують interface:

```
interface User {
 id: number;
 name: string;
 email: string;
}
function greetUser(user: User): string {
 return `Привіт, ${user.name}!`;
}
```

Різниця інтерфейсів та типів

Теорія: TypeScript

Інтерфейси є основною концепцією TypeScript і широко використовуються для визначення форм об’єктів. Вони в основному використовуються для оголошення **контрактів**, надаючи спосіб визначення форми об’єкта, якого має дотримуватися клас або інші об’єкти.

```
// Interface usage
interface Person {
 name: string;
 age: number;
}
const person: Person = {
 name: "John",
 age: 30,
};
```

Інтерфейси можна розширювати один від одного (extends), дозволяючи створювати більш спеціалізовані інтерфейси на основі існуючих.

Вони підтримують об’єднання декларацій, тобто ви можете оголосити інтерфейс у кількох місцях, а TypeScript об’єднає їх у єдине визначення.

Type Alias (Type)

Теорія: TypeScript

Аліаси типів дозволяють створити нове ім’я для певного типу. Вони використовуються для надання описової назви складним типам або для створення об’єднань, перетинів та інших складних типів. Аліаси типів можна використовувати з об’єктами, але вони більш універсальні й можуть використовуватися з іншими типами, такими як об’єднання (union), перетини (interception), тапли (typle) та примітивні типи. На відміну від інтерфейсів, **типи не можна розширити (extends)!**

```
type Person = { // Type Alias usage
 name: string;
 age: number;
};
const person: Person = {
 name: "John",
 age: 30,
};
type Status = "pending" | "in_progress" | "completed"; // Union of Type
```

Загалом, якщо потрібно створити контракт для форми об’єкта або якщо ви плануєте розширити або об’єднати визначення, інтерфейси зазвичай є кращим вибором.

З іншого боку, якщо вам потрібно створити нову назву для певного типу або працювати з об’єднаннями (union) та перетинаннями (intersection), аліаси типів забезпечують більшу гнучкість.

Псевдонім типу з типом об’єднання (union)

Теорія: TypeScript

Є функція, яка може приймати різні типи даних як вхідні дані, і ми хочемо обмежити прийнятні типи лише конкретними примітивними типами (рядок (string), число (number), логічний (boolean)). Ми можемо досягти цього, використовуючи псевдонім типу з типом об’єднання (union):

```
type Primitive = string | number | boolean;
function processData(data: Primitive) {
 // Process the data
}
// Valid usages of the function
processData("Hello");
processData(42);
processData(true);
// Invalid usage, will result in a type error
processData({ foo: "bar" });
```

Функція processData приймає лише аргументи одного з цих примітивних типів. Будь-який інший тип призведе до помилки типу під час компіляції. Якби ми використовували інтерфейс для цього сценарію, це було б невідповідним вибором, оскільки інтерфейси в основному призначені для визначення структури об’єктів, а не для створення аліасів типів для об’єднань примітивних типів.

Інтерфейси для визначення структури об’єктів

Теорія: TypeScript

Інтерфейси чудово підходять для визначення структури об’єктів і сигнатур функцій. Припустімо, ми хочемо змоделювати об’єкт користувача з певними властивостями та методом. Ми можемо зробити це за допомогою таких інтерфейсів:

```
// Interface for defining the shape of a User object
interface User {
 name: string;
 age: number;
 email: string;
 isAdmin?: boolean; // Optional property denoted by "?"
 sayHello: () => void;
}
```

У цьому прикладі інтерфейс користувача визначає необхідні властивості ім’я, вік, адресу електронної пошти та метод sayHello. Він також містить необов’язкову властивість isAdmin, позначену “?” після назви властивості.

Інтерфейси для визначення структури об’єктів

Теорія: TypeScript

```
// Implementing the User interface for a specific user
const johnDoe: User = {
 name: "John Doe",
 age: 30,
 email: "john.doe@example.com",
 isAdmin: true,
 sayHello: () => {
   console.log(`Hello, my name is ${johnDoe.name}. Nice to meet you!`);
 },
};
```

Інтерфейси для визначення структури об’єктів

Теорія: TypeScript

```
// Implementing the User interface for another user
const janeSmith: User = {
 name: "Jane Smith",
 age: 25,
 email: "jane.smith@example.com",
 sayHello: () => {
   console.log(`Hey there! I'm ${janeSmith.name}. Pleasure to meet you!`);
 },
};
```

Використовуючи інтерфейс, ми гарантуємо, що будь-який об’єкт, призначений типу User, повинен відповідати його структурі, забезпечуючи кращу безпеку типу та ясність у кодовій базі. Інтерфейс діє як угода, яку повинен виконати об’єкт користувача, що дозволяє легко зрозуміти, які властивості та методи очікуються для об’єкта користувача в додатку.
Таким чином, інтерфейси є чудовим вибором, коли вам потрібно описати форму об’єктів або сигнатури функцій, особливо під час створення чітких і добре визначених контрактів у вашому коді TypeScript.

Основні відмінності між «інтерфейсом» і «типом»

Теорія: TypeScript

Інтерфейси підтримують об’єднання декларацій, що означає, що ви можете визначити інтерфейс у кількох місцях, і TypeScript об’єднає всі декларації в одне визначення інтерфейсу. Це дозволяє розширити наявний інтерфейс додатковими властивостями або об’єднати інтерфейси з однаковою назвою.

Типи не підтримують об’єднання декларацій. Якщо ви спробуєте створити декілька аліасів типу з однаковою назвою, TypeScript викличе помилку.

```
interface User {
 name: string;
}

interface User {
 age: number;
}
// Merged User interface: { name: string; age: number; }
```

Інтерфейси можна використовувати для оголошення того, що клас реалізує певний контракт. Коли клас реалізує інтерфейс, він повинен дотримуватися структури, визначеної цим інтерфейсом.

Інтерфейси можуть розширювати інші інтерфейси

Теорія: TypeScript

Інтерфейси також можуть розширювати інші інтерфейси, успадковуючи їхні властивості та методи.

```
interface Shape {
 getArea(): number;
}
interface Circle extends Shape {
 radius: number;
}
class MyCircle implements Circle {
 radius: number;
 getArea() {
   return Math.PI * this.radius * this.radius;
 }
}
```

Аліаси типів не можна використовувати з “implements” та “extends”.

Перетинання (intersection) та об’єднання (union)

Теорія: TypeScript

Аліаси типів є більш універсальними під час роботи з типами перетинаннями (intersection) та типами об’єднаннями (union). Їх можна використовувати для створення нових типів шляхом комбінування існуючих типів.

```
type Point2D = { x: number; y: number };
type Point3D = { x: number; y: number; z: number };

type Point2DOr3D = Point2D | Point3D;
type Result = Point2D & { isValid: boolean };
```

Readability and Preference

Теорія: TypeScript

Інтерфейсам часто віддають перевагу для визначення форми об’єктів через їх явну контрактну природу. Вони самодокументуються і можуть покращити читабельність коду.
Псевдоніми типів зазвичай використовуються під час визначення складних типів або роботи з об’єднаннями та перетинаннями. Вони можуть надати більш стислі та описові назви для складних типів.

Використовуйте інтерфейси, коли:
- Ви хочете визначити структуру об’єкта або функції.
- Вам потрібно виконати об’єднання декларацій.
- Ви працюєте з командою, яка надає перевагу інтерфейсам для узгодженості.

Використовуйте аліаси типів, коли:
- Вам потрібно створити псевдонім типу для об’єднання, перетину чи інших складних типів.
- Ви працюєте з бібліотекою або стороннім кодом, який широко використовує типи.
- Ви віддаєте перевагу загальнішому підходу й не потребуєте злиття декларацій.

Readability and Preference

Теорія: TypeScript

В останніх версіях TypeScript різниця між інтерфейсами та псевдонімами типів стала менш вираженою, і вони часто використовуються як взаємозамінні. На практиці багато розробників використовують інтерфейси для більшості пов’язаних з об’єктами сценаріїв і резервують типи для більш складних маніпуляцій типами. Однак TypeScript є гнучким, і ви можете досягти подібних результатів за допомогою будь-якого підходу.

Найважливішим фактором є узгодженість вашої кодової бази. Отже, якщо у вашому проєкті вже є конвенція, загалом краще її дотримуватися. Якщо ви починаєте новий проєкт, обговоріть це рішення зі своєю командою, щоб створити послідовний підхід.

Основна відмінність між type (аліас типу) та interface (інтерфейс) полягає в тому, що interface завжди можна розширювати, додаючи нові поля пізніше, тоді як type не можна змінювати після створення.

Об'єднання декларацій (Declaration Merging)

Теорія: TypeScript

Якщо ви оголосите два інтерфейси з однаковим ім'ям, TypeScript автоматично об'єднає їх в один. З type так зробити не можна — виникне помилка дублювання.

```
//  З interface (працює):
interface User {
 name: string;
}
interface User {
 age: number; // Поля об'єднуються
}
const person: User = { name: "Іван", age: 25 };

//  З type (видасть помилку):

type User = { name: string; };
type User = { age: number; }; // Error: Duplicate identifier 'User'.
```

Можливості моделювання (Що вони можуть описувати)

Теорія: TypeScript

* interface призначений виключно для опису структури об'єктів або функцій та класів.

* type є універсальним. Він може описувати об'єкти, а також примітиви, союзи (unions), кортежі (tuples) та мапи типів.

```
// З type можна зробити так, а з interface — ні:
type ID = string | number; // Union тип
type Point = [number, number]; // Кортеж (Tuple)
type Callback = (data: string) => void; // Аліас для функції
```

Розширення (Спадкування)

Теорія: TypeScript

Обидва варіанти підтримують розширення, але синтаксис відрізняється:
* interface використовує ключове слово extends.
* type використовує оператор перетину & (Intersection).

* Використовуйте interface, коли ви описуєте структуру об'єктів публічного API, бібліотек або коли плануєте розширювати об'єкти (наприклад, для об'єктно-орієнтованого дизайну чи класів через implements).
* Використовуйте type, коли вам потрібні складні типи (союзи |, перетини &), або коли ви описуєте прості структури даних чи примітиви.

У TypeScript класи можуть реалізовувати інтерфейси за допомогою ключового слова implements. Це означає, що клас зобов'язується мати всі властивості та методи, які описані в інтерфейсі. Якщо клас не реалізує хоча б одне поле з інтерфейсу, TypeScript видасть помилку на етапі компіляції.

Спадкування інтерфейсів класами (implements)

Теорія: TypeScript

Інтерфейс виступає як "контракт" або суворий шаблон для класу. Сам інтерфейс не містить логіки (тіла методів), він лише каже, що має бути в класі.

```
// Описуємо контракт (інтерфейс)interface Printable {
 title: string;
 print(): void; // Метод, який нічого не повертає
}
// Клас реалізує інтерфейс Printableclass Book implements Printable {
 title: string;
 author: string;

 constructor(title: string, author: string) {
   this.title = title;
   this.author = author;
 }

 // Обов'язкова реалізація методу з інтерфейсу
 print(): void {
   console.log(`Книга: "${this.title}", Автор: ${this.author}`);
 }
}
// Перевірка роботиconst myBook = new Book("Кобзар", "Тарас Шевченко");
myBook.print(); // Виведе: Книга: "Кобзар", Автор: Тарас Шевченко
```

Важливі нюанси implements:

* Один клас може реалізувати кілька інтерфейсів: class User implements Identifiable, Loggable { ... } (через кому).
* implements лише перевіряє сумісність класу з інтерфейсом. Він не перевизначає типи всередині самого класу автоматично (вам все одно треба вказувати типи аргументів у методах класу).

Практика

Реалізація інтерфейсу в класі User

Реалізація інтерфейсу в класі

Практика: TypeScript

В TypeScript для реалізації інтерфейсу в класі використовується ключове слово **implements**. Клас, який імплементує інтерфейс, **зобов'язаний мати всі властивості та методи, описані в цьому інтерфейсі**.

Описуємо інтерфейс IUser

Практика: TypeScript

```
// 1. Описуємо інтерфейс IUser
interface IUser {
 id: number;
 name: string;
 email: string;
 isActive: boolean;
  // Метод без реалізації (тільки сигнатура)
 getProfileInfo(): string;
}
```

Імплементуємо інтерфейс у класі User

Практика: TypeScript

```
// 2. Імплементуємо інтерфейс у класі User
class User implements IUser {
 id: number;
 name: string;
 email: string;
 isActive: boolean;
```

РЕАЛІЗАЦІЯ КОНСТРУКТОРА

Практика: TypeScript

```
// Конструктор для ініціалізації властивостей
constructor(id: number, name: string, email: string) {
  this.id = id;
  this.name = name;
  this.email = email;
  this.isActive = true; // значення за замовчуванням
}
```

Обов'язкова реалізація методу з інтерфейсу

Практика: TypeScript

```
// Обов'язкова реалізація методу з інтерфейсу
getProfileInfo(): string {
  return `Користувач: ${this.name} (${this.email}), ID: ${this.id}`;
}

// У класі також можна створювати власні методи, яких немає в інтерфейсі
toggleStatus(): void {
  this.isActive = !this.isActive;
}
}
```

Використання класу

Практика: TypeScript

```
// 3. Використання класу
const myUser = new User(1, "Олександр", "alex@example.com");
console.log(myUser.getProfileInfo());
// Виведе: Користувач: Олександр (alex@example.com), ID: 1
```

Скорочений запис конструктора

Практика: TypeScript

Ви можете уникнути дублювання коду, якщо використаєте модифікатори доступу (наприклад, public) прямо в параметрах конструктора. TypeScript автоматично створить і присвоїть ці властивості:

```
class ShortUser implements IUser {
 isActive: boolean = true;

 constructor(
   public id: number,
   public name: string,
   public email: string
 ) {}

 getProfileInfo(): string {
   return `${this.name} - ${this.email}`;
 }
}
```

Модифікатори доступу

Практика: TypeScript

Усі властивості, описані в інтерфейсі, у самому класі обов'язково мають бути public (публічними).

Клас може реалізувати декілька інтерфейсів

Практика: TypeScript

В TypeScript клас може реалізувати декілька інтерфейсів одночасно. Для цього їхні назви просто перераховуються через кому після ключового слова implements.
Клас буде зобов'язаний виконати контракти усіх зазначених інтерфейсів.
Ось практичний приклад, де клас User реалізує два інтерфейси: IIdentifiable (для ідентифікації) та IAuthenticatable (для автентифікації).

```
// 1. Перший інтерфейс (базові дані)
interface IIdentifiable {
 id: string;
 email: string;
}

// 2. Другий інтерфейс (безпека та дії)
interface IAuthenticatable {
 role: string;
 login(): void;
 logout(): void;
}
```

Клас реалізує обидва інтерфейси через кому

Практика: TypeScript

```
class User implements IIdentifiable, IAuthenticatable { //  Клас реалізує обидва інтерфейси через кому
 // Властивості з IIdentifiable
 id: string;
 email: string;
 role: string; // Властивість з IAuthenticatable
 constructor(id: string, email: string, role: string) {
   this.id = id;
   this.email = email;
   this.role = role;
 }
 // Методи з IAuthenticatable
 login(): void {
   console.log(`Користувач ${this.email} увійшов в систему як ${this.role}.`);
 }
 logout(): void {
   console.log(`Користувач ${this.email} вийшов із системи.`);
 }
}
```

Клас реалізує обидва інтерфейси через кому

Практика: TypeScript

```
// 4. Використання класу
const admin = new User("u-99", "admin@site.com", "Admin");
admin.login(); // Виведе: Користувач admin@site.com увійшов в систему як Admin.
```

Конфлікт імен

Практика: TypeScript

Якщо два інтерфейси мають метод з однаковим ім'ям, але різними типами повернення чи параметрами, виникне помилка компіляції. Типи мають бути сумісними, або вам доведеться створити один універсальний метод у класі (через перевантаження методів), який задовольнить обидва інтерфейси.

• Альтернатива (Розширення інтерфейсів): Замість імплементації багатьох інтерфейсів у класі, іноді краще створити один третій інтерфейс, який об'єднує перші два за допомогою extends, а вже потім застосувати його до класу:

```
interface IFullUser extends IIdentifiable, IAuthenticatable {}

class User implements IFullUser { /* ... */ }
```

Методи з однаковими іменами

Практика: TypeScript

Коли клас реалізує кілька інтерфейсів, які мають методи з однаковими іменами, виникає конфлікт імен. Шлях вирішення цього конфлікту залежить від того, чи мають ці методи однакові сигнатури (типи параметрів та повернення), чи різні.

Сценарій 1: Сигнатури повністю збігаються (Найпростіший)

Якщо методи в обох інтерфейсах мають однакові параметри та повертають один і той самий тип, конфлікту для TypeScript немає. Вам достатньо написати метод у класі всього один раз. Він автоматично задовольнить контракти обох інтерфейсів.

Обидва інтерфейси вимагають метод save()

Практика: TypeScript

```
interface ILocalSaver {
 save(): void;
}

interface ICloudSaver {
 save(): void;
}

// Обидва інтерфейси вимагають метод save(): void
class BackupManager implements ILocalSaver, ICloudSaver {
 // Одна реалізація покриває обидва інтерфейси
 save(): void {
   console.log("Дані збережено і локально, і в хмару!");
 }
}
```

Перевантаження методів

Практика: TypeScript

Сценарій 2: Сигнатури відрізняються (Перевантаження методів)

Якщо методи мають однакову назву, але приймають різні типи параметрів або повертають різні значення, виникне помилка компіляції. Щоб її вирішити, у класі потрібно використати перевантаження методів (method overloading): спочатку оголосити сигнатури методів з інтерфейсів, а потім написати один універсальний метод, який може обробити всі варіанти.
Приклад з класом User:

```
interface IEmailNotifier {
 // Очікує текст повідомлення, повертає boolean (успішно чи ні)
 notify(message: string): boolean;
}

interface ISmsNotifier {
 // Очікує код помилки у вигляді числа, нічого не повертає
 notify(code: number): void;
}
```

Єдина універсальна реалізація методу

Практика: TypeScript

```
class User implements IEmailNotifier, ISmsNotifier {
 // 1. Оголошуємо сигнатури для сумісності з інтерфейсами (без тіла методу)
 notify(message: string): boolean;
 notify(code: number): void;
 // 2. Єдина універсальна реалізація методу
 // Тип параметра об'єднуємо через union (string | number)
 // Тип повернення об'єднуємо або використовуємо any
 notify(payload: string | number): any {
   if (typeof payload === "string") {
     console.log(`Надсилаємо Email: ${payload}`);
     return true; // задовольняє IEmailNotifier
   } else if (typeof payload === "number") {
     console.log(`Надсилаємо SMS з кодом помилки: ${payload}`);
     // повертає undefined за замовчуванням, що задовольняє void в ISmsNotifier
   }
 }
}
```

Використання

Практика: TypeScript

```
// Використання:
const user = new User();
const emailResult = user.notify("Привіт!"); // Працює як функція з string -> boolean
user.notify(404);    // Працює як функція з number -> void
```

Альтернатива

Практика: TypeScript

Альтернатива: Що робити, якщо логіка методів має бути кардинально різною?

Якщо метод save() з першого сценарію повинен робити зовсім різні речі для хмари та для локального диска, реалізувати їх в одному класі під одним ім'ям не вийде, бо вони заважатимуть один одному.

В такому випадку краще змінити архітектуру:
1. Перейменувати методи в самих інтерфейсах (saveToLocal() та saveToCloud()).
2. Або замість імплементації обох інтерфейсів в одному класі, створити два окремі класи (наприклад, LocalUserSaver та CloudUserSaver), кожен з яких реалізує свій інтерфейс.

Інтерфейс з опціональними членами

Практика: TypeScript

Для створення опціональних (необов'язкових) властивостей або методів в інтерфейсах TypeScript використовується знак питання ? після імені властивості.
Це означає, що клас, який реалізує цей інтерфейс, може, але не зобов'язаний мати цю властивість або метод. Тип такої властивості автоматично стає тип | undefined.
Ось детальний приклад із класом User:

```
// 1. Описуємо інтерфейс з опціональними членами
interface IUser {
 id: number;
 name: string;
  // Опціональні властивості
 age?: number;
 phoneNumber?: string;

 // Опціональний метод
 sendNotification?(): void;
}
```

Клас реалізує інтерфейс

Практика: TypeScript

```
class User implements IUser {
 id: number;
 name: string;
 age?: number;  // Ми вирішили реалізувати лише одну опціональну властивість
 // phoneNumber та sendNotification ми вирішили взагалі не додавати в цей клас
 constructor(id: number, name: string, age?: number) {
   this.id = id;
   this.name = name;
   if (age) {
     this.age = age;
   }
 }
}
// 3. Створення об'єктів
const userWithAge = new User(1, "Іван", 25);
const userWithoutAge = new User(2, "Марія");
console.log(userWithAge.age);    // Виведе: 25
console.log(userWithoutAge.age); // Виведе: undefined
```

Опціональна властивість може бути undefined

Практика: TypeScript

Коли ви працюєте з об'єктом через інтерфейс, TypeScript знає, що опціональна властивість може бути undefined. Тому ви не можете викликати її методи або властивості напряму без перевірки. Неправильно (виникне помилка компіляції):

```
function printAge(user: IUser) {
 // Помилка: Об'єкт можливо має значення 'undefined'
 console.log(user.age.toFixed());
}
```

Безпечний виклик опціонального методу

Практика: TypeScript

```
// Правильно (безпечна робота):
// Використовуйте оператор опціонального ланцюжка (?.) або стандартну перевірку:

function printAge(user: IUser) {
 // 1. За допомогою ?. (якщо age є, викличеться toFixed(), якщо немає — поверне undefined)
 console.log(user.age?.toFixed());

 // 2. Через класичну перевірку if
 if (user.age !== undefined) {
   console.log(user.age.toFixed());
 }
}
// Безпечний виклик опціонального методу:
function notify(user: IUser) {
 user.sendNotification?.(); // Викличеться лише якщо метод існує
}
```

Модифікатор readonly в інтерфейсах

Практика: TypeScript

Модифікатор readonly в інтерфейсах TypeScript використовується для створення властивостей, які можна прочитати, але не можна змінити після їхньої першої ініціалізації. Це чудовий спосіб захистити дані об'єкта (наприклад, унікальні ідентифікатори або системні налаштування) від випадкового перезапису.
Ось практичний приклад використання readonly у класі User:
readonly діє лише на рівні компіляції TypeScript: Після компіляції коду в чистий JavaScript (JS) модифікатор readonly зникає, оскільки в JS немає вбудованої підтримки readonly-властивостей для класів у такий спосіб. Проте під час розробки TypeScript надійно заблокує будь-яку спробу змінити це поле.

```
// 1. Описуємо інтерфейс із незмінними властивостями
interface IUser {
 readonly id: string;       // Змінити id після створення буде неможливо
 readonly createdAt: Date;  // Дата створення також захищена
 name: string;              // Звичайне поле, яке можна змінювати
}
```

Модифікатор readonly в інтерфейсах

Практика: TypeScript

```
// 2. Клас реалізує інтерфейс
class User implements IUser {
 // У класі ми також можемо продублювати readonly для додаткового захисту
 readonly id: string;
 readonly createdAt: Date;
 name: string;

 constructor(id: string, name: string) {
   this.id = id;
   this.createdAt = new Date(); // Ініціалізація дозволена лише в конструкторі
   this.name = name;
 }
}
```

Модифікатор readonly в інтерфейсах

Практика: TypeScript

```
// 3. Тестуємо роботу модифікатора
const user = new User("usr-404", "Дмитро");

user.name = "Олексій"; // ✅ Дозволено (звичайне поле)

// ❌ ПОМИЛКА КОМПІЛЯЦІЇ: Cannot assign to 'id' because it is a read-only property.
// user.id = "usr-555";

// ❌ ПОМИЛКА КОМПІЛЯЦІЇ: Cannot assign to 'createdAt' because it is a read-only property.
// user.createdAt = new Date();
```

Скорочений запис у конструкторі

Практика: TypeScript

Скорочений запис у конструкторі: Якщо ви використовуєте скорочений запис конструктора, ви можете вказати readonly безпосередньо там:

```
class ShortUser implements IUser {
 readonly createdAt: Date = new Date();

 // TypeScript автоматично створить readonly-властивість 'id'
 constructor(readonly id: string, public name: string) {}
}
```

Об'єкти та масиви всередині readonly

Практика: TypeScript

Якщо властивість readonly є об'єктом або масивом, сам об'єкт/масив перезаписати не можна, але його вміст змінювати можна.

```
interface ITeam {
 readonly members: string[];
}


const team: ITeam = { members: ["Петро"] };
// team.members = ["Іван"]; // ❌ Помилка! Перезапис масиву заборонено.
team.members.push("Анна");    // ✅ Дозволено! Сам масив не змінився, ми лише додали елемент.
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Typescriptlang: https://www.typescriptlang.org/

Tsconfig: https://www.typescriptlang.org/tsconfig/

Microsoft for JavaScript developers: https://developer.microsoft.com/en-us/javascript/

JavaScript Guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide

Nodejs:](https://nodejs.org/uk)

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua